

Zhi-Hua Zhou

Machine Learning

Translated by Shaowu Liu

The slides of this chapter
are translated by Xin-Chun Li



Springer

Chapter 10

Dimension Reduction and Metric Learning

Content

- ❑ **k-Nearest Neighbor Learning**
- ❑ **Low-dimensional Embedding**
- ❑ **Principal Component Analysis**
- ❑ **Manifold Learning**
- ❑ **Metric Learning**

Content

- ❑ **k-Nearest Neighbor Learning**
- ❑ Low-dimensional Embedding
- ❑ Principal Component Analysis
- ❑ Manifold Learning
- ❑ Metric Learning

k-Nearest Neighbor Learning

Introduction of k-Nearest Neighbor Learning

- k-Nearest Neighbor (kNN) learning is a common supervised learning method:
 - Obtain training samples and a specific distance metric;
 - Given a testing sample, find the closest k samples in training set, take the voting of these samples for classification or averaging for regression. Weighted voting or averaging can also be utilized: the smaller the distance, the larger the weight.
 - Voting: predict the testing sample as the most frequent class label in the k neighbors
 - Averaging: predict the testing sample as the average of the k real-valued outputs

Lazy Learning and Eager Learning

k-Nearest Neighbor Learning does not seem to have an explicit training process, and it belongs to lazy learning

- ❑ Lazy learning: simply stores the samples in the training phase and does nothing until the testing samples are received
- ❑ Eager learning: learn from samples in the training phase

Illustration of k-Nearest Neighbor Learning

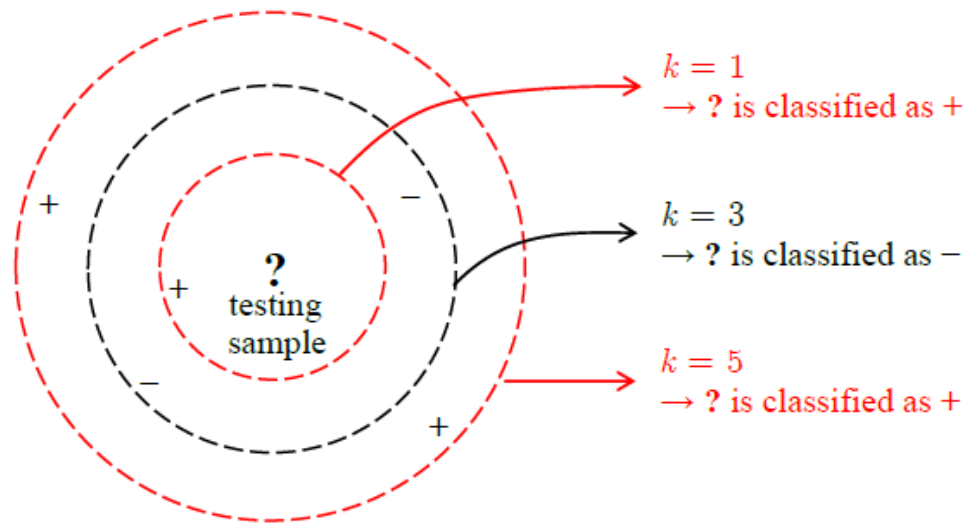


Fig. 10.1: The k NN classifier. The dashed lines are equidistant lines; the testing sample is classified as positive when $k = 1$ or $k = 5$ and is classified as negative when $k = 3$.

- We can see that the parameter k plays an important role here since different k values may lead to very different classification results. In addition, different distance calculations may also lead to significantly different neighborhood hence different classification results.

k-Nearest Neighbor Learning

Binary classification error of 1NN

- Suppose that we have selected a proper distance, that is, k appropriate neighbors can be identified. Then, let us take a look at the use of One-Nearest Neighbor Classifier (1NN with $k = 1$) for binary classification problems.
- Given a testing sample x , suppose its nearest neighbor is z , the misclassification rate of 1NN is the probability that x and z have different class labels, i.e.,

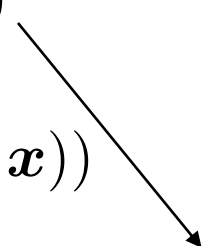
$$P(err) = 1 - \sum_{c \in \mathcal{Y}} P(c|x)P(c|z)$$

Probability that x and z
have the same class c

k-Nearest Neighbor Learning

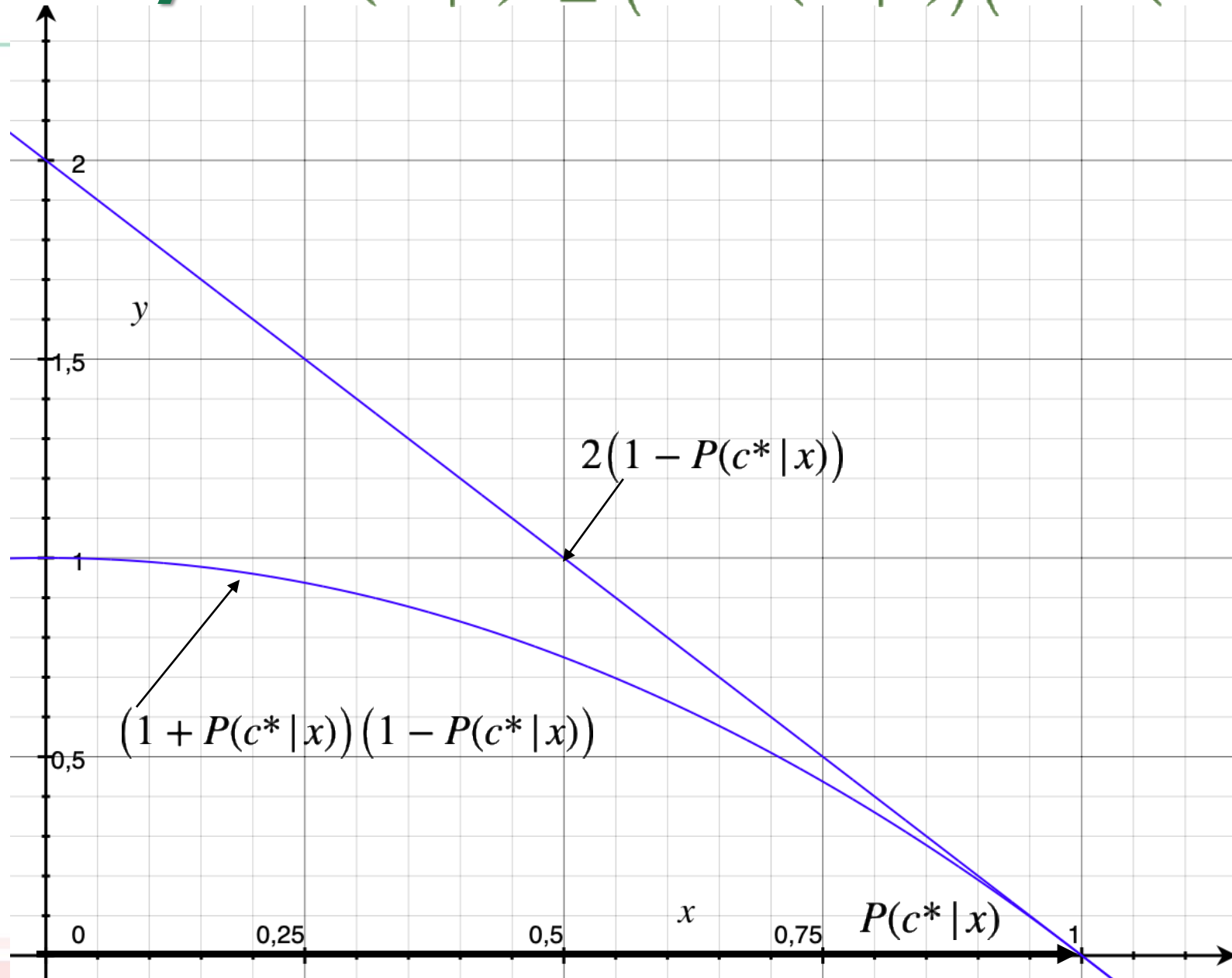
Binary classification error of 1NN

- Suppose that the samples are *i.i.d.* and also we can always find a sample within an arbitrarily small positive range δ for any $\mathbf{x}$. In other words, for any testing samples, the training sample can always be found within an arbitrarily small range.
- Let $c^* = \arg \max_{c \in \mathcal{Y}} P(c|\mathbf{x})$ denotes the result of the Bayes optimal classifier, then

$$\begin{aligned} P(err) &= 1 - \sum_{c \in \mathcal{Y}} P(c|\mathbf{x})P(c|\mathbf{z}) \simeq 1 - \sum_{c \in \mathcal{Y}} P^2(c|\mathbf{x}) \\ &\leq 1 - P^2(c^*|\mathbf{x}) = (1 + P(c^*|\mathbf{x}))(1 - P(c^*|\mathbf{x})) \\ &\leq 2 \times (1 - P(c^*|\mathbf{x})). \end{aligned}$$

$$\sum_{\forall c \in \mathcal{Y}} P^2(c|\mathbf{x}) \geq P^2(c^*|\mathbf{x})$$

- Though the 1NN classifier is simple, its generalization error is at most twice of the error of the Bayes optimal classifier!

Why $1 - P(c^* | x)^2 \leq (1 + P(c^* | x))(1 - P(c^* | x))$?



Content

- k-Nearest Neighbor Learning
- **Low-dimensional Embedding**
- Principal Component Analysis
- Manifold Learning
- Metric Learning

Low-dimensional Embedding

Curse of Dimensionality

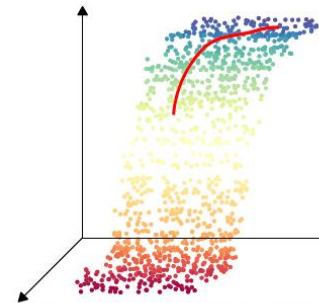
- ❑ Our discussions in the previous section rely on one important assumption: we can always find a sample within an arbitrarily small positive range δ for any x , which means the sampling must be sufficiently dense, *a.k.a.* dense sampling:
 - Suppose $\delta = 0.001$ and there is only one feature, we need 1000 samples uniformly distributed within the normalized value range of the given feature. However, this is only the case when the dimension is 1. If the dimension increases, the situation will change significantly. For instance, given 20 features, we would need $(10^3)^{20} = 10^{60}$ samples to satisfy the dense sampling requirement (*).
 - Since the number of features can easily go beyond thousands in practice, it is generally impractical to fulfil the dense sampling requirement. Besides, distance calculations, as required in many learning methods, become difficult in high-dimensional space. For example, even the inner product is no longer easy in high-dimensional cases.
 - High dimensionality creates issues, such as data sparsity and the difficulty of distance calculation, that are faced by all machine learning methods. This phenomenon is known as the curse of dimensionality.

(*) Notice: the number of elementary particles in the observable universe is about 10^{80}

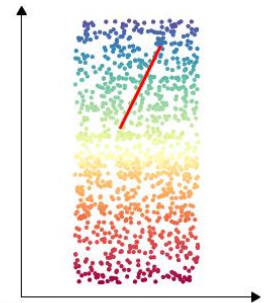
Low-dimensional Embedding

A general approach to alleviating the curse of dimensionality is **dimensionality reduction**

- Converting the original high-dimensional feature space into a **low dimensional subspace** via **mathematical transformations (embedding)**.
- Why it is possible to perform dimensionality reduction?
- In many cases, although the samples observed or collected by people are high-dimensional, **what is closely related to the learning task may be only a low-dimensional distribution**, that is, there is a low-dimensional embedding in the high-dimensional space.



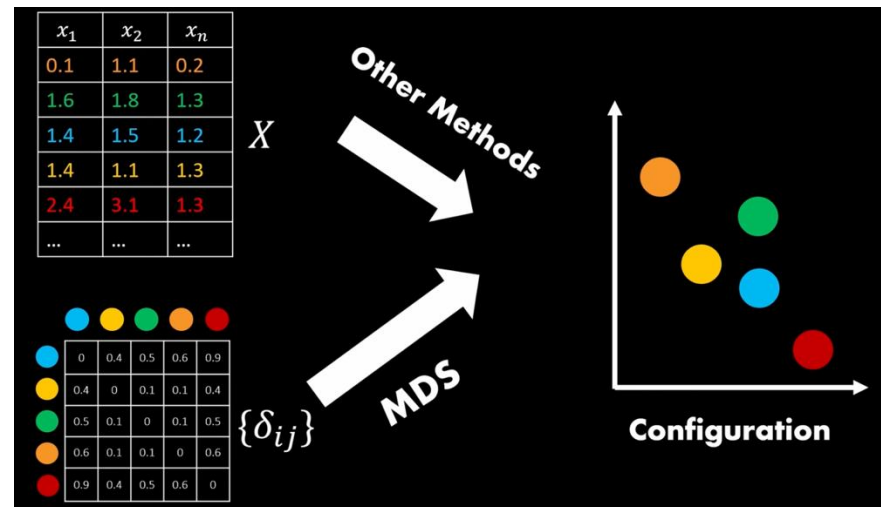
(a) Observation in 3-dimensional space.



(b) Observation in 2-dimensional space.

Multiple Dimensional Scaling

- ❑ Multiple Dimensional Scaling (MDS) projects a data set from the original space into a lower-dimensional space while preserving the distances between samples
- ❑ It is a class of techniques and dates back to a paper of 1952 [Togerson]
- ❑ Embeddings in LLM are a new version
- ❑ It is available in scikit learn
- ❑ It is based on three steps:
 1. Computes the GRAM matrix= $X^T X$
 2. Eigenvectors decomposition: keeps the d^* larger eigenvalues
 3. Projection in the space of the eigenvectors to obtain the final data matrix $\tilde{X}$



$X^T X$ contains the objects correlations

$$X = \begin{pmatrix} X_{11} & \cdots & X_{m1} \\ \vdots & \ddots & \vdots \\ X_{1n} & \cdots & X_{mn} \end{pmatrix}$$

Diagram: An oval encircles the first column of matrix X (elements $X_{11}, \vdots, X_{1n}$). An arrow points from the text "1st object" to this oval.

$$X^T = \begin{pmatrix} X_{11} & \cdots & X_{1n} \\ \vdots & \ddots & \vdots \\ X_{m1} & \cdots & X_{mn} \end{pmatrix}$$

Diagram: An oval encircles the first row of matrix X^T (elements $X_{11}, \cdots, X_{1n}$). An arrow points from the text "1st object" to this oval.

$$X^T X = \begin{pmatrix} X_1 \cdot X_1 & \cdots & \cdots & X_1 \cdot X_m \\ X_2 \cdot X_1 & X_2 \cdot X_2 & \cdots & X_2 \cdot X_m \\ \cdots & \cdots & \cdots & \cdots \\ X_m \cdot X_1 & X_m \cdot X_2 & \cdots & X_m \cdot X_m \end{pmatrix}$$

Diagram: Two arrows point from the text blocks below to the element $X_m \cdot X_1$ in the matrix.

Each term of $X^T X$ aggregates a pair of objects for all the n features

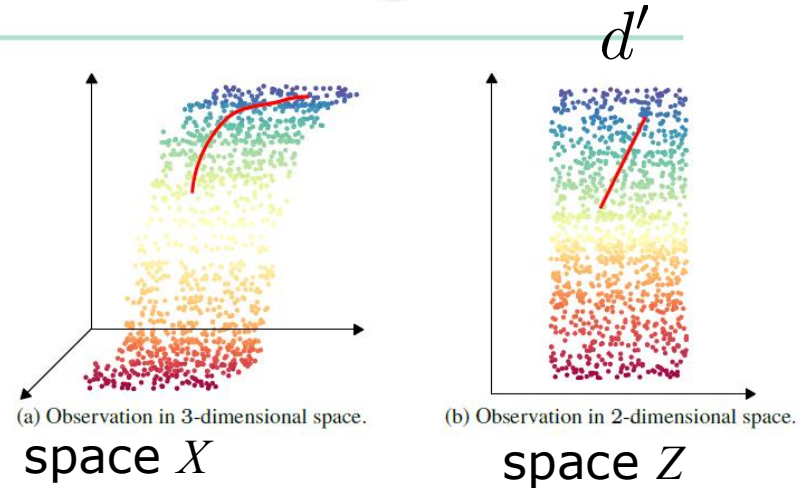
It is the scalar product between vector x_m and vector x_1

The Gram matrix contains the similarity measures between objects; for centered data, it is proportional to the covariance matrix.

Multiple Dimensional Scaling

- Let $D \in \mathbb{R}^{m \times m}$ be a distance matrix of m samples in the original space, where the distance between the samples x_i and x_j is $dist_{ij}$.

$$D = \begin{bmatrix} dist_{11} & dist_{12} & \dots & dist_{1m} \\ dist_{21} & dist_{22} & \dots & dist_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ dist_{m1} & dist_{m2} & \dots & dist_{mm} \end{bmatrix}$$

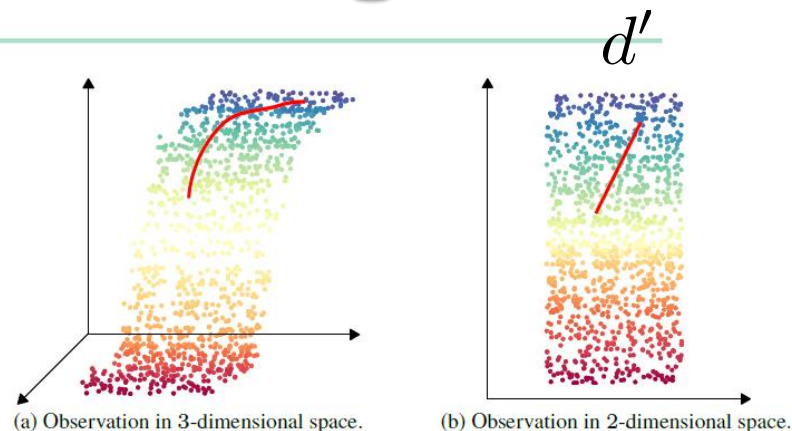


$$dist_{ij} = x_i - x_j = \langle x_{i1} | \dots x_{id} \rangle - \langle x_{j1} | \dots x_{jd} \rangle = \langle x_{i1} - x_{j1} | \dots x_{id} - x_{jd} \rangle$$

- The goal of subsequent slides is to find a relationship between the distances (D , in the original space X) and the correlation matrix (B) in the new space (Z)

Multiple Dimensional Scaling

- The task is to obtain the sample representation in the d' -dimensional space, and the distance between any two samples is the same in both spaces, i.e., $\|z_i - z_j\| = \text{dist}_{ij}$.
- Let $\mathbf{B} = \mathbf{Z}^T \mathbf{Z} \in \mathbb{R}^{m \times m}$ be the inner product matrix of the projected samples
- $b_{ij} = z_i^T z_j$ denotes the inner product of the i -th and j -th projected sample



$$\mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1m} \\ b_{21} & b_{22} & \dots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \dots & b_{mm} \end{bmatrix}$$

$$\left. \begin{array}{l} z_i = \langle z_{i1} | \dots | z_{id'} \rangle \\ z_j = \langle z_{j1} | \dots | z_{jd'} \rangle \end{array} \right\} \begin{bmatrix} z_{i1} \\ z_{i2} \\ \vdots \\ z_{id'} \end{bmatrix} \cdot \begin{bmatrix} z_{j1} & z_{j2} & \dots & z_{jd'} \end{bmatrix} = \sum_{k=1}^{d'} z_{ik} \cdot z_{jk} = b_{ij}$$

proportional to the correlation between z_i and z_j (with zero mean)

Multiple Dimensional Scaling

□ Let $\mathbf{B} = \mathbf{Z}^T \mathbf{Z} \in \mathbb{R}^{m \times m}$ be the inner product matrix of the projected samples:

$b_{ij} = \mathbf{z}_i^T \mathbf{z}_j$ denotes the inner product of the i -th and j -th projected sample

$$\mathbf{z}_i = \langle z_{i1} | \dots | z_{id'} \rangle$$

$$\mathbf{z}_j = \langle z_{j1} | \dots | z_{jd'} \rangle$$

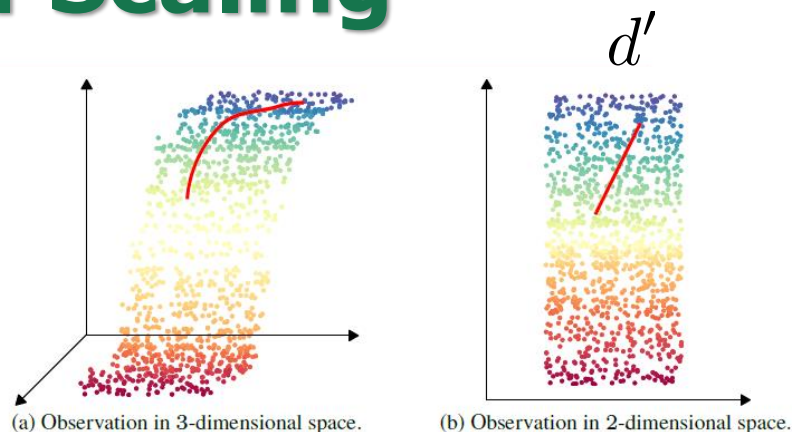
□ We have:

$$\begin{aligned} dist_{ij}^2 &= ||\mathbf{z}_i||^2 + ||\mathbf{z}_j||^2 - 2\mathbf{z}_i^T \mathbf{z}_j \\ &= b_{ii} + b_{jj} - 2b_{ij}. \end{aligned}$$

because:

$$dist_{ij} = \mathbf{z}_i - \mathbf{z}_j = \langle z_{i1} | \dots | z_{id'} \rangle - \langle z_{j1} | \dots | z_{jd'} \rangle = \langle z_{i1} - z_{j1} | \dots | z_{id'} - z_{jd'} \rangle$$

$$\begin{aligned} dist_{ij}^2 &= ||\langle z_{i1} - z_{j1} | \dots | z_{id'} - z_{jd'} \rangle||^2 = (z_{i1} - z_{j1})^2 + \dots + (z_{id'} - z_{jd'})^2 = \\ &= (z_{i1}^2 + z_{j1}^2 - 2z_{i1}z_{j1}) + \dots + (z_{id'}^2 + z_{jd'}^2 - 2z_{id'}z_{jd'}) \end{aligned}$$



$$\mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1m} \\ b_{21} & b_{22} & \dots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \dots & b_{mm} \end{bmatrix}$$

Multiple Dimensional Scaling

- We assume the projected samples $\mathbf{Z}$ are **zero-centered**, i.e., $\sum_{i=0}^m \mathbf{z}_i = 0$. Then, we can know that the summations of the columns and rows of the inner product matrix $\mathbf{B}$ are:

$$\sum_{i=1}^m b_{ij} = \sum_{j=1}^m b_{ij} = 0.$$

(zero centered means the mean value is equal to zero and the sum is equal to zero)

Why? Because:

$$b_{ij} = \sum_{k=1}^{d'} z_{ik} \cdot z_{jk} = z_{i1} \cdot z_{j1} + \dots + z_{id'} \cdot z_{jd'}$$

$$\Rightarrow b_{ij} = b_{ji}$$

because matrix $\mathbf{B}$ is symmetrical

$$\begin{aligned} \sum_{i=1}^m b_{ij} &= z_{11} \cdot z_{j1} + \dots + z_{1d'} \cdot z_{jd'} + \\ &\quad z_{21} \cdot z_{j1} + \dots + z_{2d'} \cdot z_{jd'} + \\ &\quad \dots + \\ &\quad z_{m1} \cdot z_{j1} + \dots + z_{md'} \cdot z_{jd'} \end{aligned}$$

Multiple Dimensional Scaling

- We assume the projected samples $\mathbf{Z}$ are **zero-centered**, i.e., $\sum_{i=0}^m \mathbf{z}_i = 0$. Then, we can know that the summations of the columns and rows of the inner product matrix $\mathbf{B}$ are:

$$\sum_{i=1}^m b_{ij} = \sum_{j=1}^m b_{ij} = 0.$$

Why? Because:

$$b_{ij} = \sum_{k=1}^{d'} z_{ik} \cdot z_{jk} = z_{i1} \cdot z_{j1} + \cdots + z_{id'} \cdot z_{jd'}$$

$$\begin{aligned} \sum_{i=1}^m b_{ij} &= z_{11} \cdot z_{j1} + \cdots + z_{1d'} \cdot z_{jd'} + \\ &\quad z_{21} \cdot z_{j1} + \cdots + z_{2d'} \cdot z_{jd'} + \\ &\quad \cdots + \\ &\quad z_{m1} \cdot z_{j1} + \cdots + z_{md'} \cdot z_{jd'} \\ &= (z_{11} + z_{21} + \cdots + z_{m1}) \cdot z_{j1} + \\ &\quad (z_{11} + z_{21} + \cdots + z_{m1}) \cdot z_{j2} + \\ &\quad \cdots + \\ &\quad (z_{11} + z_{21} + \cdots + z_{m1}) \cdot z_{jd'} \end{aligned}$$

by reordering the terms

Multiple Dimensional Scaling

- We assume the projected samples Z are **zero-centered**, i.e., $\sum_{i=1}^m z_i = 0$. Then, we can know that the summations of the columns and rows of the inner product matrix B are:

$$\sum_{i=1}^m b_{ij} = \sum_{j=1}^m b_{ij} = 0.$$

Why? Because:

$$b_{ij} = \sum_{k=1}^{d'} z_{ik} \cdot z_{jk} = z_{i1} \cdot z_{j1} + \cdots + z_{id'} \cdot z_{jd'}$$

$$\begin{aligned} \sum_{i=1}^m b_{ij} &= z_{11} \cdot z_{j1} + \cdots + z_{1d'} \cdot z_{jd'} + \\ &\quad z_{21} \cdot z_{j1} + \cdots + z_{2d'} \cdot z_{jd'} + \\ &\quad \cdots + \\ &\quad z_{m1} \cdot z_{j1} + \cdots + z_{md'} \cdot z_{jd'} \\ &= (z_{11} + z_{21} + \cdots + z_{m1}) \cdot z_{j1} + \\ &\quad (z_{11} + z_{21} + \cdots + z_{m1}) \cdot z_{j2} + \\ &\quad \cdots + \\ &\quad (z_{11} + z_{21} + \cdots + z_{m1}) \cdot z_{jd'} \\ &= 0 \end{aligned}$$

Multiple Dimensional Scaling

□ Denote $\text{tr}(\cdot)$ as the trace of a matrix.

$$\text{tr}(\mathbf{B}) = b_{11} + b_{22} + \dots + b_{mm}$$

$$\mathbf{B} = \begin{bmatrix} \boxed{b_{11}} & b_{12} & & b_{1m} \\ b_{21} & \boxed{b_{22}} & & b_{2m} \\ \vdots & \vdots & \boxed{\phantom{b_{mm}}} & \vdots \\ b_{m1} & b_{m1} & & \boxed{b_{mm}} \end{bmatrix}$$

We can obtain the following equations:

$$\sum_{i=1}^m \text{dist}_{ij}^2 = \text{tr}(\mathbf{B}) + mb_{jj}, \quad \sum_{j=1}^m \text{dist}_{ij}^2 = \text{tr}(\mathbf{B}) + mb_{ii}, \quad \sum_{i=1}^m \sum_{j=1}^m \text{dist}_{ij}^2 = 2m \text{tr}(\mathbf{B}),$$

$$\sum_{i=1}^m (\text{tr}(\mathbf{B}) + mb_{ii}) = m \cdot \text{tr}(\mathbf{B}) + m \cdot \text{tr}(\mathbf{B})$$

Why? Remember that: $\text{dist}_{ij}^2 = \|\mathbf{z}_i\|^2 + \|\mathbf{z}_j\|^2 - 2\mathbf{z}_i^T \mathbf{z}_j$

$$= b_{ii} + b_{jj} - 2b_{ij}.$$

Summation goes to 0

In the summation it becomes

Multiple Dimensional Scaling

- Then we could obtain the inner product matrix **B** from distance matrix **D** via the distance-preserved property:

$$dist_{i.}^2 = \frac{1}{m} \sum_{j=1}^m dist_{ij}^2 \quad dist_{.j}^2 = \frac{1}{m} \sum_{i=1}^m dist_{ij}^2 \quad dist_{..}^2 = \frac{1}{m^2} \sum_{i=1}^m \sum_{j=1}^m dist_{ij}^2$$

$$b_{ij} = -\frac{1}{2}(dist_{ij}^2 - dist_{i.}^2 - dist_{.j}^2 + dist_{..}^2).$$

Why? Because:

$$\begin{aligned} dist_{ij}^2 &= \|z_i\|^2 + \|z_j\|^2 - 2z_i^T z_j \\ &= b_{ii} + b_{jj} - 2b_{ij}. \end{aligned} \quad \Rightarrow \quad b_{ij} = -\frac{1}{2}(dist_{ij}^2 - b_{ii} - b_{jj})$$

where we substitute b_{ii} , b_{jj} and $tr(\mathbf{B})$ from:

$$b_{ii} = \frac{1}{m}(\sum_j dist_{ij}^2 - tr(\mathbf{B})) \quad b_{jj} = \frac{1}{m}(\sum_i dist_{ij}^2 - tr(\mathbf{B}))$$

$$tr(\mathbf{B}) = \frac{1}{2m} \sum_i \sum_j dist_{ij}^2$$

Multiple Dimensional Scaling

- We calculate the eigenvalue decomposition of matrix $\mathbf{B}$, i.e., $\mathbf{B} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$.
- $\mathbf{\Lambda} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$ is the diagonal matrix with the eigenvalues.
- $\mathbf{V}$ is the eigenvector matrix.
- We assume there are d^* non-zero eigenvalues.
- The corresponding diagonal matrix and eigenvector matrix are $\mathbf{\Lambda}_* = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_{d^*})$ and $\mathbf{V}_*$
- Then (projection), $\mathbf{Z}$ could be decomposed as:

$$\mathbf{Z} = \mathbf{\Lambda}_*^{1/2} \mathbf{V}_*^T \in \mathbb{R}^{d^* \times m}$$

- In practice, we only require the dimension-reduced distances to be close to the original distances rather than the same.
- In such cases, we can take the $d' \ll d$ largest eigenvalues.
- We make a diagonal matrix $\tilde{\mathbf{\Lambda}} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_{d'})$
- We denote $\tilde{\mathbf{V}}$ as the corresponding eigenvector matrix.
- Then, we have:

$$\mathbf{Z} = \tilde{\mathbf{\Lambda}}^{1/2} \tilde{\mathbf{V}}^T \in \mathbb{R}^{d' \times m}.$$

Eigenvalue decomposition of B

$$\mathbf{B} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T$$

Main property of eigenvectors: $\mathbf{B} \cdot \mathbf{v}_i = \lambda_i \mathbf{v}_i$

If in addition, they are orthogonal:

$$\mathbf{v}_i \cdot \mathbf{v}_j = 0, i \neq j$$

$$\mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & b_{1m} \\ b_{21} & b_{22} & b_{2m} \\ \vdots & \vdots & \vdots \\ b_{m1} & b_{m2} & b_{mm} \end{bmatrix}$$

$$\mathbf{V} = \begin{bmatrix} v_{11} & v_{12} & v_{1d} \\ v_{21} & v_{22} & v_{2d} \\ \vdots & \vdots & \vdots \\ v_{m1} & v_{m2} & v_{md} \end{bmatrix}$$

$$\mathbf{\Lambda} = \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \vdots \\ 0 & 0 & \lambda_{d*} \end{bmatrix}$$

$$\mathbf{V} \mathbf{\Lambda} = \begin{bmatrix} v_{11}\lambda_1 & v_{12}\lambda_2 & v_{1d}\lambda_{d*} \\ v_{21}\lambda_1 & v_{22}\lambda_2 & v_{2d}\lambda_{d*} \\ \vdots & \vdots & \vdots \\ v_{m1}\lambda_1 & v_{m2}\lambda_2 & v_{md}\lambda_{d*} \end{bmatrix}$$

$$\mathbf{V}^T = \begin{bmatrix} v_{11} & v_{12} & v_{m1} \\ v_{21} & v_{22} & v_{m2} \\ \vdots & \vdots & \vdots \\ v_{1d*} & v_{2d*} & v_{md*} \end{bmatrix}$$

$$\mathbf{V} \mathbf{\Lambda} \mathbf{V}^T = \begin{bmatrix} \sum_j v_{1j}^2 \lambda_j & \sum_j v_{1j} v_{2j} \lambda_j & \dots \\ \sum_j v_{2j} v_{1j} \lambda_j & \sum_j v_{2j}^2 \lambda_j & \\ \dots & & \\ \sum_j v_{mj} v_{1j} \lambda_j & \dots & \sum_j v_{mj}^2 \lambda_j \end{bmatrix}$$

=0 for terms out of the diagonal
for orthogonal eigenvectors

Multiple Dimensional Scaling

MDS Algorithm

Algorithm 10.1 Multiple Dimensional Scaling.

Input: Distance matrix $\mathbf{D} \in \mathbb{R}^{m \times m}$, where $dist_{ij}$ is the distance from $\mathbf{x}_i$ to $\mathbf{x}_j$;
Dimension d' of the low-dimensional space.

Process:

- 1: Compute $dist_{i.}^2$, $dist_{.j}^2$, and $dist_{..}^2$ according to $dist_{i.}^2 = \frac{1}{m} \sum_{j=1}^m dist_{ij}^2$, $dist_{.j}^2 = \frac{1}{m} \sum_{i=1}^m dist_{ij}^2$, $dist_{..}^2 = \frac{1}{m^2} \sum_{i=1}^m \sum_{j=1}^m dist_{ij}^2$.
- 2: Compute inner product matrix $\mathbf{B}$ according to $b_{ij} = -\frac{1}{2}(dist_{ij}^2 - dist_{i.}^2 - dist_{.j}^2 + dist_{..}^2)$.
- 3: Perform eigenvalue decomposition on matrix $\mathbf{B}$;
- 4: Take the d' largest eigenvalues to make a diagonal matrix $\tilde{\mathbf{\Lambda}}$, and $\tilde{\mathbf{V}}$ is the corresponding eigenvector matrix.

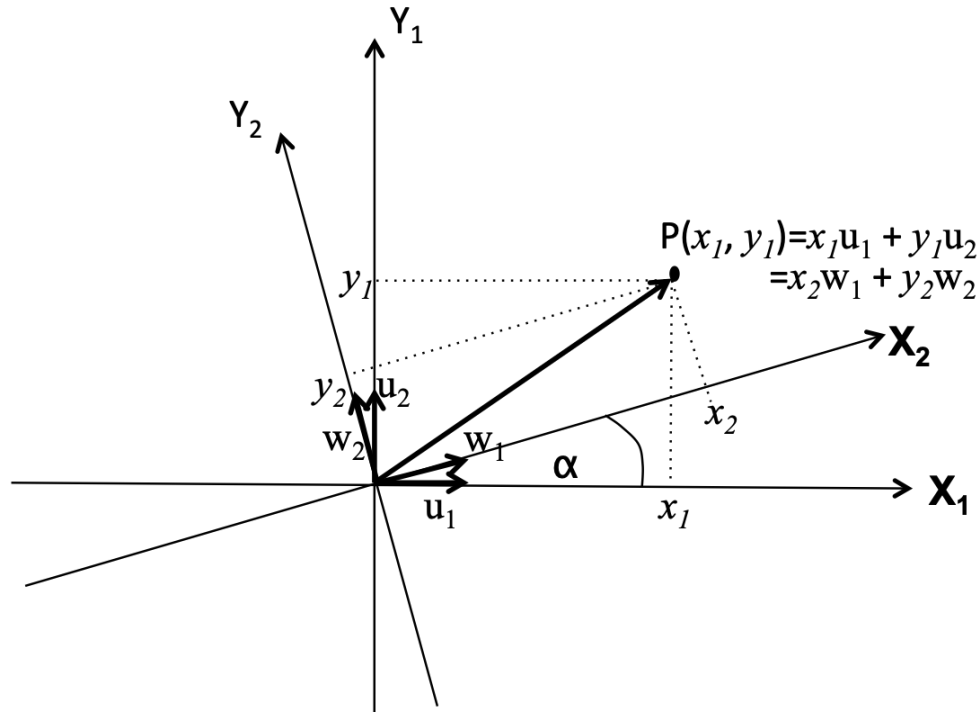
Output: The matrix $\tilde{\mathbf{V}}\tilde{\mathbf{\Lambda}}^{1/2} \in \mathbb{R}^{m \times d'}$, where each row gives the coordinates of the sample in the low-dimensional space.

↙
Transposition of $\mathbf{Z} = \tilde{\mathbf{\Lambda}}^{1/2}\tilde{\mathbf{V}}^T \in \mathbb{R}^{d' \times m}$.

Linear Dimension Reduction

- In general, the simplest method to obtain a lower-dimensional subspace is applying linear transformations to the original high-dimensional space.
- Given the samples $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m) \in \mathbb{R}^{d \times m}$ in a d -dimensional space, the transformed samples in the d' -dimensional space are:
 $\mathbf{Z} = \mathbf{W}^T \mathbf{X}$,
where $\mathbf{W} \in \mathbb{R}^{d \times d'}$ is the transformation matrix,
 $\mathbf{Z} \in \mathbb{R}^{d' \times m}$ is the representations in the low-dimensional space.
- The transformation matrix $\mathbf{W}$ could be viewed as d' basis vectors with a dimension d .
- $\{\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{d'}\}$ could be seen as a new coordinate system.
- If $\mathbf{w}_i$ and $\mathbf{w}_j (i \neq j)$ are orthogonal ($\mathbf{w}_i^T \cdot \mathbf{w}_j = 0$), the new coordinate system is an orthogonal coordinate system, and $\mathbf{W}$ is an orthogonal transformation matrix.
- The methods that perform dimensionality reduction by linear transformations are called linear dimensionality reduction.

Summary on matrix-vector multiplication, rotation and projection



$$\begin{pmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{pmatrix} \cdot \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} m_{11} \cdot v_1 + m_{12} \cdot v_2 \\ m_{21} \cdot v_1 + m_{22} \cdot v_2 \end{pmatrix}$$

$$W = \begin{pmatrix} w_1 & w_2 \\ \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}$$

$$W \mathbf{u} = \mathbf{w}$$

$$\begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \cdot \begin{pmatrix} u_1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} w_1 \\ \cos \alpha \\ \sin \alpha \end{pmatrix}$$

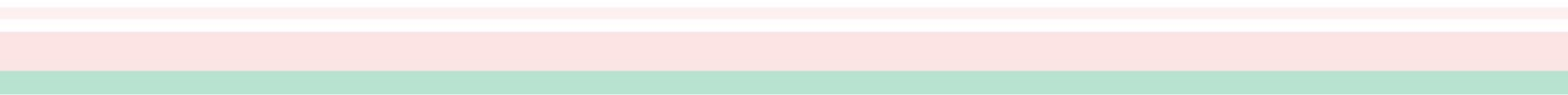
$$\begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \cdot \begin{pmatrix} u_2 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} w_2 \\ -\sin \alpha \\ \cos \alpha \end{pmatrix}$$

In order to find x_2 and y_2 we have to project the vector defining P onto w_1 and w_2 :

$$x_2 = \mathbf{w}_1 \cdot \mathbf{p} = \langle \cos(\alpha)u_1, \sin(\alpha)u_2 \rangle \cdot \langle x_1u_1, y_1u_2 \rangle = x_1 \cos(\alpha) + y_1 \sin(\alpha)$$

$$y_2 = \mathbf{w}_2 \cdot \mathbf{p} = \langle -\sin(\alpha)u_1, \cos(\alpha)u_2 \rangle \cdot \langle x_1u_1, y_1u_2 \rangle = -x_1 \sin(\alpha) + y_1 \cos(\alpha)$$

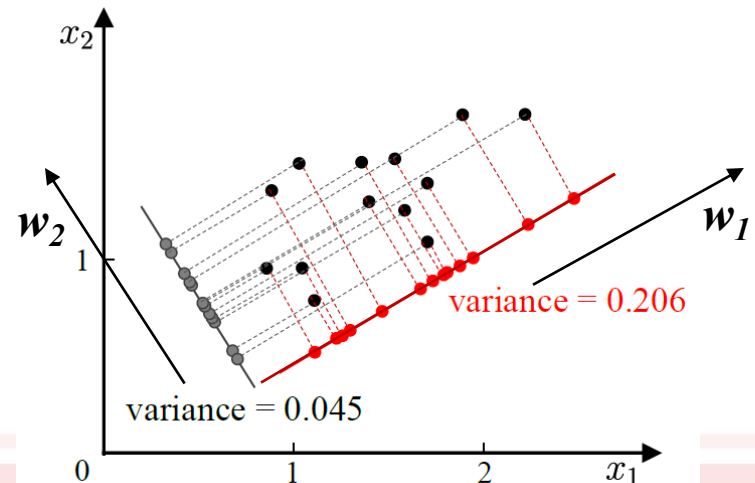
Content

- k-Nearest Neighbor Learning
 - Low-dimensional Embedding
 - **Principal Component Analysis**
 - Manifold Learning
 - Metric Learning
- 

Principal Component Analysis

Principal Component Analysis (PCA)

- ❑ For the samples in an orthogonal feature space, how can we use a hyperplane to represent the samples?
- ❑ Intuitively, if such a hyperplane exists, then it perhaps needs to have the following properties:
 1. Minimum reconstruction error: the samples should have short distances to this hyperplane;
 2. Maximum variance: the projections of samples onto the hyperplane should stay away from each other.
- ❑ The above two properties lead to two equivalent derivations of PCA.



Principal Component Analysis

1. Minimum reconstruction error

- Centering the sample matrix, $\sum_i \mathbf{x}_i = 0$, and the obtained new coordinate system is $\{\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_d\}$.
 $\mathbf{w}_i$ is an orthonormal basis vector, satisfying

$$\|\mathbf{w}_i\|_2 = 1, \mathbf{w}_i^T \mathbf{w}_j = 0 (i \neq j).$$

- We discard some dimensions in the new coordinate system, reducing the dimension to $d' < d$, then we have the projection $\mathbf{z}_i = (z_{i1}; z_{i2}; \dots; z_{id'})$.
 $z_{ij} = \mathbf{w}_j^T \mathbf{x}_i$ is the j -th dimension coordinate via projecting $\mathbf{x}_i$.

We reconstruct $\mathbf{x}_i$ from $\mathbf{z}_i$:

$$\hat{\mathbf{x}}_i = \sum_{j=1}^{d'} z_{ij} \mathbf{w}_j.$$

Principal Component Analysis

1. Minimum reconstruction error

- Consider all samples, the distance of the raw samples $\mathbf{x}_i$ and the reconstructed samples $\hat{\mathbf{x}}_i$ is:

$$\sum_{i=1}^m \left\| \sum_{j=1}^{d'} z_{ij} \mathbf{w}_j - \mathbf{x}_i \right\|_2^2 = \sum_{i=1}^m \mathbf{z}_i^T \mathbf{z}_i - 2 \sum_{i=1}^m \mathbf{z}_i^T \mathbf{W}^T \mathbf{x}_i + \text{const}$$
$$\propto -\text{tr} \left(\mathbf{W}^T \left(\sum_{i=1}^m \mathbf{x}_i \mathbf{x}_i^T \right) \mathbf{W} \right).$$

- We should minimize the above equation.
Due to $\mathbf{w}_j$ is an orthonormal basis vector, and $\sum \mathbf{x}_i \mathbf{x}_i^T$ is (proportional to) the covariance matrix, we have:

$$\begin{aligned} \min_{\mathbf{W}} \quad & -\text{tr}(\mathbf{W}^T \mathbf{X} \mathbf{X}^T \mathbf{W}) \\ \text{s.t.} \quad & \mathbf{W}^T \mathbf{W} = \mathbf{I}. \end{aligned}$$

of the zero centered data

The above is the optimization target of PCA.

Principal Component Analysis

2. Maximum variance

- Sample $\mathbf{x}_i$ corresponds to a projected point $\mathbf{W}^T \mathbf{x}_i$.
If we wish the projections of samples stay away from each other, then the variance of the projected samples should be maximized.
The covariance matrix of projected samples is $\sum_i \mathbf{W}^T \mathbf{x}_i \mathbf{x}_i^T \mathbf{W}$, then we have:

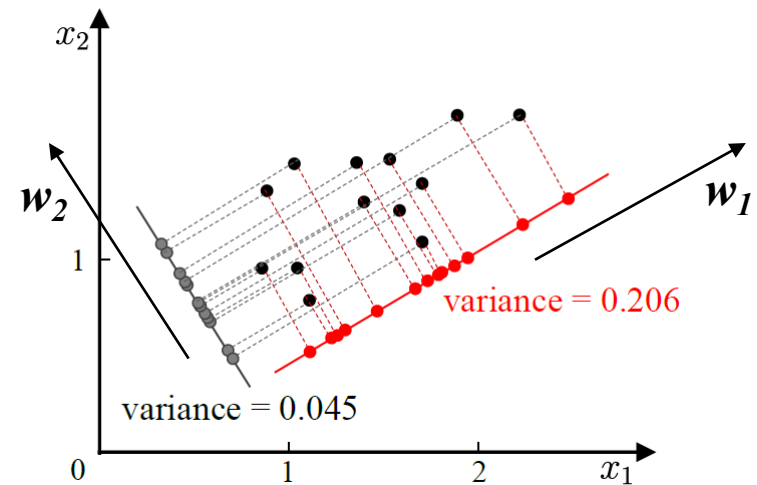
$$\max_{\mathbf{W}} \quad \text{tr}(\mathbf{W}^T \mathbf{X} \mathbf{X}^T \mathbf{W})$$

$$\text{s.t.} \quad \mathbf{W}^T \mathbf{W} = \mathbf{I}.$$

This problem is equivalent to:

$$\min_{\mathbf{W}} \quad -\text{tr}(\mathbf{W}^T \mathbf{X} \mathbf{X}^T \mathbf{W})$$

$$\text{s.t.} \quad \mathbf{W}^T \mathbf{W} = \mathbf{I}.$$



Principal Component Analysis

Solving PCA

$$\begin{aligned} \max_{\mathbf{W}} \quad & \text{tr}(\mathbf{W}^T \mathbf{X} \mathbf{X}^T \mathbf{W}) \\ \text{s.t.} \quad & \mathbf{W}^T \mathbf{W} = \mathbf{I}. \end{aligned}$$

$$\begin{aligned} \min_{\mathbf{W}} \quad & -\text{tr}(\mathbf{W}^T \mathbf{X} \mathbf{X}^T \mathbf{W}) \\ \text{s.t.} \quad & \mathbf{W}^T \mathbf{W} = \mathbf{I}. \end{aligned}$$

□ Applying Lagrange multipliers to the above equations:

$$\mathbf{X} \mathbf{X}^T \mathbf{w} = \lambda_i \mathbf{w} \quad (\text{eq. 10.17})$$

We perform eigenvalue decomposition on the covariance matrix $\mathbf{X} \mathbf{X}^T$ and obtain the sorted eigenvalues: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$. Then, we construct the solution using the eigenvectors of the first d' eigenvalues, i.e., $\mathbf{W}^* = (\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{d'})$.

Principal Component Analysis

PCA Algorithm

Algorithm 10.2 Principal Component Analysis.

Input: Data set $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$;
Dimension d' of the lower-dimensional space.

Process:

- 1: Center all samples: $\mathbf{x}_i \leftarrow \mathbf{x}_i - \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i$;
- 2: Compute the covariance matrix $\mathbf{X}\mathbf{X}^T$ of samples;
- 3: Perform eigenvalue decomposition on the covariance matrix $\mathbf{X}\mathbf{X}^T$;
- 4: Take the eigenvectors $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{d'}$ corresponding to the d' largest eigenvalues.

Output: The projection matrix $\mathbf{W}^* = (\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{d'})$.

Principal Component Analysis

- Typically, the dimension of the lower-dimensional space is specified by the user or selected by doing cross-validation with different d' values, that is, comparing the performance of a kNN classifier in the dimension-reduced spaces produced by different d' values.

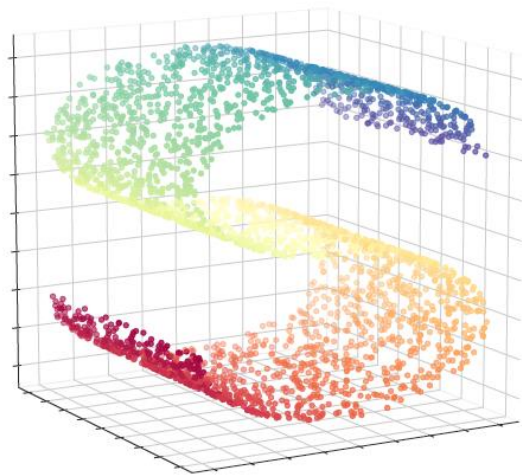
For PCA, a threshold t of the retained variance (e.g., $t = 95\%$) can also be set from the reconstruction perspective to find the minimum d' subject to:

$$\frac{\sum_{i=1}^{d'} \lambda_i}{\sum_{i=1}^d \lambda_i} \geq t.$$

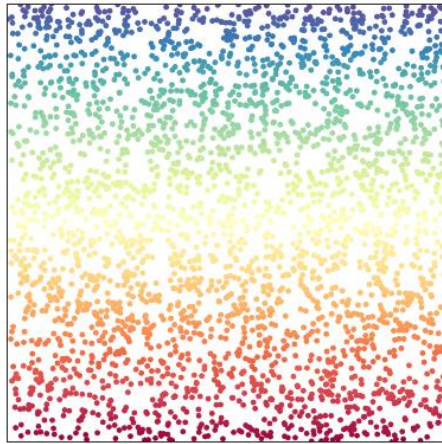
- With just $\mathbf{W}^*$ and the mean vector of samples, PCA can project new samples to the lower-dimensional space by applying simple vector subtraction and matrix-vector multiplication operations.
- Reducing the dimensionality:
 - produces an information loss
 - can lead to relatively denser sampling;
 - can reduce the noise: the eigenvectors of the smallest eigenvalues often relate to the noises: discarding these dimensions helps in noise reduction.

Kernelized Linear Dimension Reduction

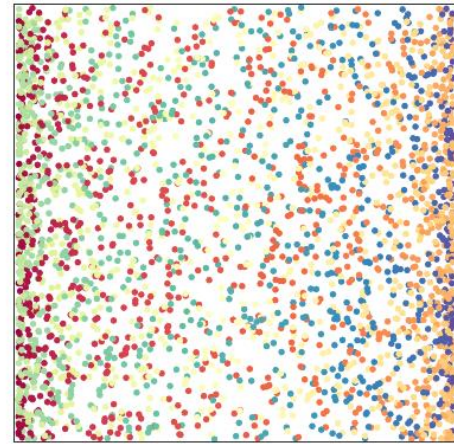
- Linear dimensionality reduction methods transform a high-dimensional space into a low-dimensional space via a linear mapping. In practice, however, nonlinear mappings are often needed to find the proper low-dimensional embedding:



(a) Observation in the 3-dimensional space.



(b) Intrinsic 2-dimensional structure.



(c) Result of PCA dimensionality reduction.

Fig (a) shows an embedding of data points sampled from a squared region shown in Fig (b) to an S-shaped surface in a three-dimensional space. PCA results (in Fig (c)) loses the original low-dimensional structure.

Kernelized Linear Dimension Reduction

Kernelized PCA (KPCA)

- A general approach to non-linear dimensionality reduction is to *kernelize* linear dimensionality reduction methods via kernel tricks.
- Suppose that we project data from the high-dimensional feature space to a hyperplane spanned by $\mathbf{W} = (\mathbf{w}_1, \dots, \mathbf{w}_d)$. Then for $\mathbf{w}_j$, from the PCA objective function of eq. (10.17) with Lagrange multipliers, we have:

$$\left(\sum_{i=1}^m \mathbf{z}_i \mathbf{z}_i^T \right) \mathbf{w}_j = \lambda_j \mathbf{w}_j$$

- $\mathbf{z}_i$ is the image of $\mathbf{x}_i$ in the high-dimensional feature space.

Then, we have:

$$\mathbf{w}_j = \frac{1}{\lambda_j} \left(\sum_{i=1}^m \mathbf{z}_i \mathbf{z}_i^T \right) \mathbf{w}_j = \sum_{i=1}^m \mathbf{z}_i \frac{\mathbf{z}_i^T \mathbf{w}_j}{\lambda_j} = \sum_{i=1}^m \mathbf{z}_i \alpha_i^j.$$

α_i^j is the j -th component of the vector $\boldsymbol{\alpha}_i$ (proportional to the projection of $\mathbf{z}_i$ on $\mathbf{w}_j$)

Kernelized Linear Dimension Reduction

Kernelized PCA (KPCA)

- Suppose that $\mathbf{z}_i$ is obtained by mapping the original sample $\mathbf{x}_i$ via ϕ :

$$\mathbf{z}_i = \phi(\mathbf{x}_i), \quad i = 1, 2, \dots, m.$$

- From the previous slide we have: $\mathbf{W} = \sum_{i=1}^m \mathbf{z}_i \alpha_i$

- If the explicit form of ϕ is known, then we can use it to map the samples to the high-dimensional feature space, and then apply PCA, i.e.,

with

$$\text{funzione kernel} \left(\sum_{i=1}^m \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T \right) \mathbf{w}_j = \lambda_j \mathbf{w}_j$$

$$\mathbf{w}_j = \sum_{i=1}^m \phi(\mathbf{x}_i) \alpha_i^j.$$

Kernelized Linear Dimension Reduction

Kernelized PCA (KPCA)

- The exact form of ϕ is generally unknown.
Hence, we introduce the kernel function:

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j).$$

- Substituting $\mathbf{w}_j = \sum_{i=1}^m \phi(\mathbf{x}_i) \alpha_i^j$, into $\left(\sum_{i=1}^m \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T \right) \mathbf{w}_j = \lambda_j \mathbf{w}_j$ gives:

$$\mathbf{K} \boldsymbol{\alpha}^j = \lambda_j \boldsymbol{\alpha}^j.$$

- $\mathbf{K}$ is the kernel matrix corresponding to κ with:

$$(\mathbf{K})_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j), \boldsymbol{\alpha}^j = (\alpha_1^j; \dots; \alpha_m^j).$$

- The above is an **eigenvalue decomposition problem**:
hence, we simply take the eigenvectors corresponding to the d' largest eigenvalues in $\mathbf{K}$.

Kernelized Linear Dimension Reduction

Kernelized PCA (KPCA)

- For a new sample $\mathbf{x}$, its projected coordinate in the j -th dimension is z_j

$$z_j = \mathbf{w}_j^T \phi(\mathbf{x})$$

Substituting $\mathbf{w}_j = \sum_{i=1}^m \phi(\mathbf{x}_i) \alpha_i^j$, in z_j , we have:

$$\begin{aligned} z_j &= \sum_{i=1}^m \alpha_i^j \underbrace{\phi(\mathbf{x}_i)^T \phi(\mathbf{x})}_{\kappa(\mathbf{x}_i, \mathbf{x})} \\ &= \sum_{i=1}^m \alpha_i^j \kappa(\mathbf{x}_i, \mathbf{x}) \quad \text{where } \alpha_i \text{ has been normalized.} \end{aligned}$$

- We could see that KPCA is computationally expensive since it **sums over all samples** to compute the projected coordinate.

Content

- k-Nearest Neighbor Learning
- Low-dimensional Embedding
- Principal Component Analysis
- **Manifold Learning**
- Metric Learning

Manifold Learning

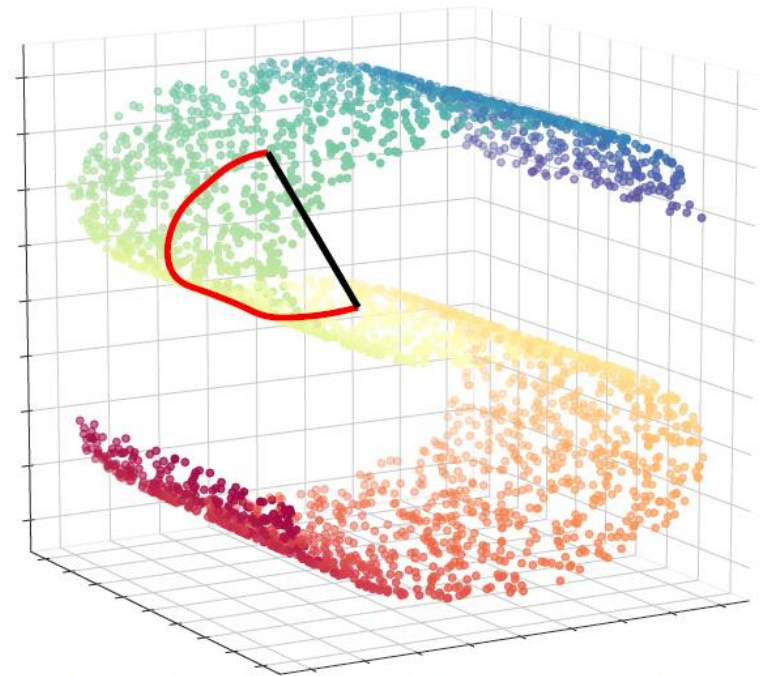
- ❑ Manifold learning is a dimensionality reduction approach that utilizes some concepts of topological manifolds.
- ❑ A manifold is a topological space that is locally homeomorphic to the Euclidean space.
It locally meets the properties of Euclidean space, which means we can use Euclidean distance.
- ❑ If a **low-dimensional manifold is embedded in the high-dimensional space, then the samples, which seem to be very complex in the high-dimensional space, locally have the same properties of samples in the Euclidean space.**
Therefore, we can establish dimensionality reduction mappings locally, and then extend to the entire space.
- ❑ When the dimensionality is reduced to 2 or 3, we can naturally visualize the data, and hence manifold learning is also useful for visualization purposes.

Manifold Learning

Isometric Mapping (Isomap)

The straight-line Euclidean distances measured in a high-dimensional space can be misleading since the straight lines in the high-dimensional space may not exist in the low-dimensional manifold embedding.

The **geodesic distance** is the intrinsic distance between the two points.

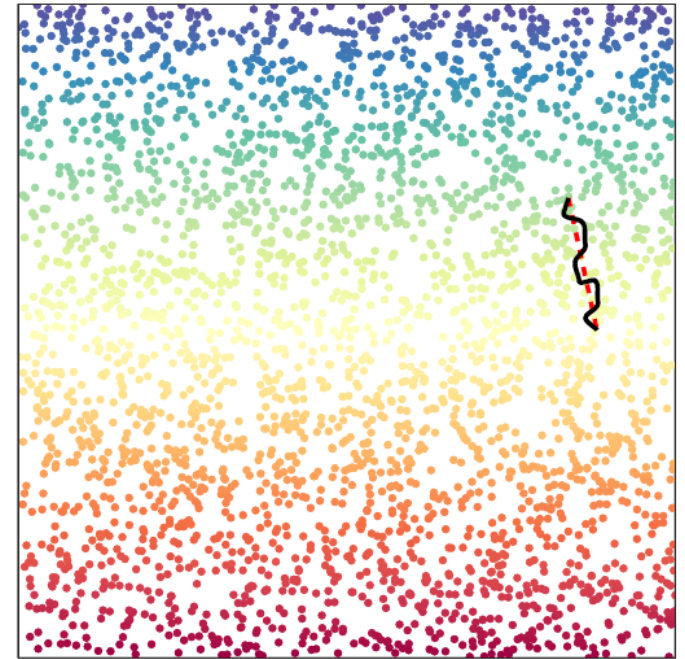


(a) The geodesic distance and the straight-line distance.

Manifold Learning

Isometric Mapping (Isomap)

- A manifold is locally homeomorphic to the Euclidean space, and we can find neighboring points of each point based on the Euclidean distance.
- Hence, a neighborhood graph can be constructed. In the neighborhood graph, only neighboring points are connected; the **geodesic distance is measured by finding the shortest path** between two points in the neighborhood graph.
- To find the shortest path between two points in a neighborhood graph, we can use the well-known Dijkstra algorithm or the Floyd algorithm.



(b) The geodesic distance and the neighborhood distance.

Manifold Learning

manifold--> traduzione ideale : varietà

Isometric Mapping (Isomap)

Algorithm 10.3 Isometric Mapping.

Input: Data set $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$;
Nearest neighbor parameter k ;
Dimension d' of the low-dimensional space.

Process:

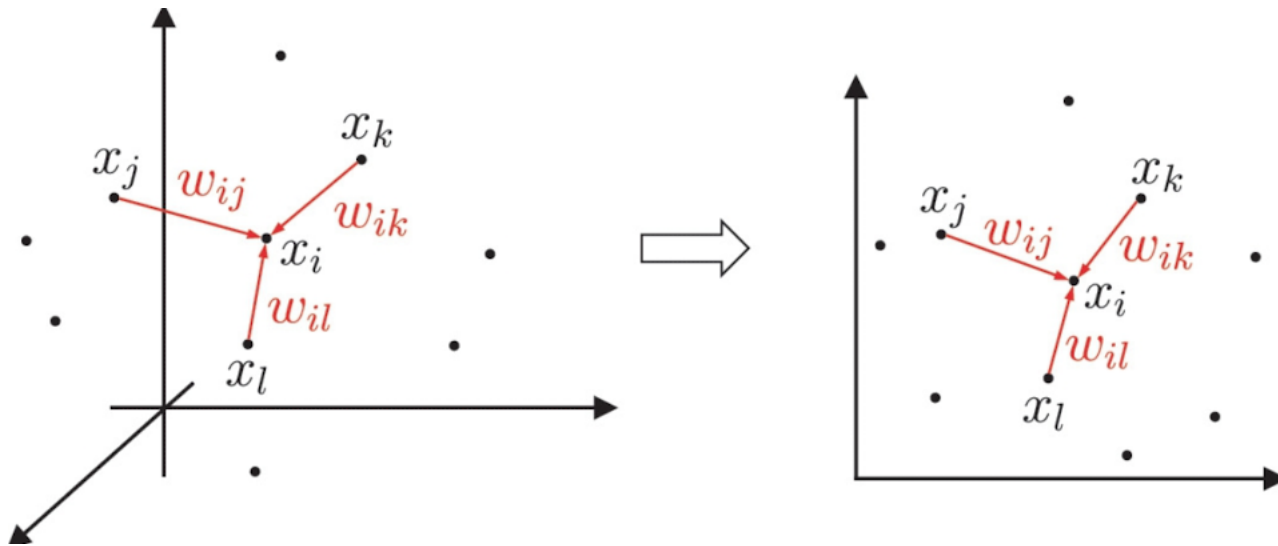
- 1: **for** $i = 1, 2, \dots, m$ **do**
- 2: Find the k nearest neighbors of $\mathbf{x}_i$;
- 3: Set the distances between $\mathbf{x}_i$ and the k nearest neighbors to the Euclidean distance, and set the distances from $\mathbf{x}_i$ to other sample points as positive infinity;
- 4: **end for**
- 5: Compute the distance $\text{dist}(\mathbf{x}_i, \mathbf{x}_j)$ between every pair sample points using a shortest path algorithm;
- 6: Use $\text{dist}(\mathbf{x}_i, \mathbf{x}_j)$ as input for the MDS algorithm;
- 7: **return** The output of the MDS algorithm.

Output: The low-dimensional projections $Z = \{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_m\}$ of the data set D .

Manifold Learning

Locally Linear Embedding (LLE)

- Locally Linear Embedding (LLE) aims to keep the linear relationships between neighboring samples.



$$\mathbf{x}_i = w_{ij}\mathbf{x}_j + w_{ik}\mathbf{x}_k + w_{il}\mathbf{x}_l$$

Manifold Learning

Locally Linear Embedding (LLE)

- LLE starts by identifying the neighborhood indices set Q_i for sample $\mathbf{x}_i$
- Then find the linear reconstruction weights w_{ij} of samples in Q_i :

The goal is setting $\mathbf{w}_i \rightarrow \min_{\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_m} \sum_{i=1}^m \left\| \mathbf{x}_i - \sum_{j \in Q_i} w_{ij} \mathbf{x}_j \right\|_2^2$ Eq. (10.27)

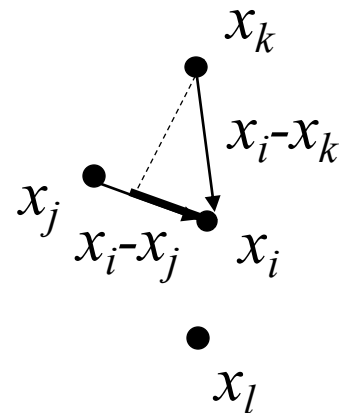
s.t. $\sum_{j \in Q_i} w_{ij} = 1,$

$\mathbf{x}_i$ and $\mathbf{x}_j$ are known.

Letting $C_{jk} = (\mathbf{x}_i - \mathbf{x}_j)^T (\mathbf{x}_i - \mathbf{x}_k)$

Then w_{ij} has a closed form solution:

$$w_{ij} = \frac{\sum_{k \in Q_i} C_{jk}^{-1}}{\sum_{l, s \in Q_i} C_{ls}^{-1}} \begin{matrix} \longrightarrow \text{keeping fixed } x_j \\ \longrightarrow \text{for all pairs of} \\ \text{neighbours } x_l \text{ and } x_s \end{matrix}$$



Manifold Learning

Locally Linear Embedding (LLE)

- Since LLE preserves w_i in the low-dimensional space, the low-dimensional coordinates z_i of x_i can be obtained by:

The goal is setting z_i $\rightarrow$ $\min_{z_1, z_2, \dots, z_m} \sum_{i=1}^m \left\| z_i - \sum_{j \in Q_i} w_{ij} z_j \right\|_2^2$ Eq. (10.29)

- Let $\mathbf{Z} = (z_1; z_2; \dots; z_m) \in \mathbb{R}^{d' \times m}$, $(\mathbf{W})_{ij} = w_{ij}$,

$$\mathbf{M} = (\mathbf{I} - \mathbf{W})^T (\mathbf{I} - \mathbf{W}), \quad \text{matrix form of Eq. (10.29)}$$

- The optimization can be rewritten as follows and solved by eigenvalue decomposition (in $\mathbf{M}$) and eigenvectors (in $\mathbf{Z}$):

$$\begin{aligned} \min_{\mathbf{Z}} \operatorname{tr}(\mathbf{Z} \mathbf{M} \mathbf{Z}^T) \\ \text{s.t. } \mathbf{Z} \mathbf{Z}^T = \mathbf{I}. \end{aligned}$$

Manifold Learning

Locally Linear Embedding (LLE)

Algorithm 10.4 Locally Linear Embedding.

Input: Data set $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$;
Nearest neighbor parameter k ;
Dimension d' of the low-dimensional space.

Process:

1: **for** $i = 1, 2, \dots, m$ **do**

2: Find the k nearest neighbors of $\mathbf{x}_i$;

3: Compute w_{ij} using (10.27) for $j \in Q_i$;

4: Set $w_{ij} = 0$ for $j \notin Q_i$;

5: **end for**

6: Obtain $\mathbf{M}$ using $(\mathbf{M} = (\mathbf{I} - \mathbf{W})^T(\mathbf{I} - \mathbf{W}))$,

7: Perform eigenvalue decomposition on $\mathbf{M}$;

8: **return** the eigenvectors corresponding to the d' smallest eigenvalues of $\mathbf{M}$.

Output: The low-dimensional projections $Z = \{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_m\}$ of the data set D .

$$\begin{aligned} \min_{w_1, w_2, \dots, w_m} \sum_{i=1}^m \left\| \mathbf{x}_i - \sum_{j \in Q_i} w_{ij} \mathbf{x}_j \right\|_2^2 \\ \text{s.t. } \sum_{j \in Q_i} w_{ij} = 1, \end{aligned}$$

Content

- k-Nearest Neighbor Learning
- Low-dimensional Embedding
- Principal Component Analysis
- Manifold Learning
- **Metric Learning**

Metric Learning

Motivation

- ❑ In machine learning, the main purpose of dimensionality reduction is to find a lower-dimensional space, in which the learning performance is better than that in the original high-dimensional space.
- ❑ Since each space corresponds to a distance metric defined on the sample features, the searching of an appropriate space is indeed searching for an appropriate distance metric.
- ❑ Then, why not learn the appropriate distance metric directly?

Metric Learning

- For two d -dimensional samples $\mathbf{x}_i$ and $\mathbf{x}_j$, their squared Euclidean distance can be written as:

$$\text{dist}_{\text{ed}}^2(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2^2 = \text{dist}_{ij,1}^2 + \text{dist}_{ij,2}^2 + \cdots + \text{dist}_{ij,d}^2,$$

- $\text{dist}_{ij,k}$ is the distance between $\mathbf{x}_i$ and $\mathbf{x}_j$ on the k -th dimension. Suppose different features have different importance, then we can introduce the feature weights $\mathbf{w}$:

$$\begin{aligned} \text{dist}_{\text{wed}}^2(\mathbf{x}_i, \mathbf{x}_j) &= \|\mathbf{x}_i - \mathbf{x}_j\|_2^2 = w_1 \cdot \text{dist}_{ij,1}^2 + w_2 \cdot \text{dist}_{ij,2}^2 + \cdots + w_d \cdot \text{dist}_{ij,d}^2 \\ &= (\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{W} (\mathbf{x}_i - \mathbf{x}_j), \end{aligned}$$

- $w_i \geq 0$, $\mathbf{W} = \text{diag}(\mathbf{w})$ is a diagonal matrix, $(\mathbf{W})_{ii} = w_i$, and the adjustable weights are the parameters to be learned from data.

Metric Learning

- We notice that the non-diagonal elements of $\mathbf{W}$ are zero, which means the axes are orthogonal. In practice, however, this is often not true. Hence, by replacing $\mathbf{W}$ with a symmetric positive semidefinite matrix $\mathbf{M}$, we have the Mahalanobis distance:

$$\text{dist}_{\text{mah}}^2(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{W} (\mathbf{x}_i - \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_{\mathbf{M}}^2,$$

where $\mathbf{M}$ is also called the metric matrix, and metric learning is about learning $\mathbf{M}$. In order to ensure the distances are non-negative and symmetric, $\mathbf{M}$ must be a positive semidefinite matrix, that is, there must be a matrix $\mathbf{P}$ such that $\mathbf{M} = \mathbf{P}\mathbf{P}^T$.

- To learn $\mathbf{M}$, we need to set an objective.
- Suppose we wish to improve the performance of the nearest neighbor classifier, then $\mathbf{M}$ can be directly embedded into the classifier's performance measure, and the optimal $\mathbf{M}$ is obtained by optimizing the performance measure.

Positive semi-definite matrices $\mathbf{M}$ are defined such that the scalars $\mathbf{x}^T \mathbf{M} \mathbf{x}$ and $\mathbf{z}^* \mathbf{M} \mathbf{z}$ are non negative for any vector $\mathbf{x}$ with positive values.

Metric Learning

Neighbourhood Component Analysis (NCA)

- Nearest neighbor classifiers usually employ the voting method for classification, that is, each neighboring sample has 1 vote, and each non-neighboring sample has 0 vote.
- Instead of using binary voting, we can also use the probabilistic voting method, in which the probability of a sample $\mathbf{x}_j$ influences the classification of a sample $\mathbf{x}_i$ is given by:

$$p_{ij} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|_{\mathbf{M}}^2)}{\sum_l \exp(-\|\mathbf{x}_i - \mathbf{x}_l\|_{\mathbf{M}}^2)},$$

- p_{ij} is maximized when $i = j$. The impact of $\mathbf{x}_j$ on $\mathbf{x}_i$ decreases as their distance increases.

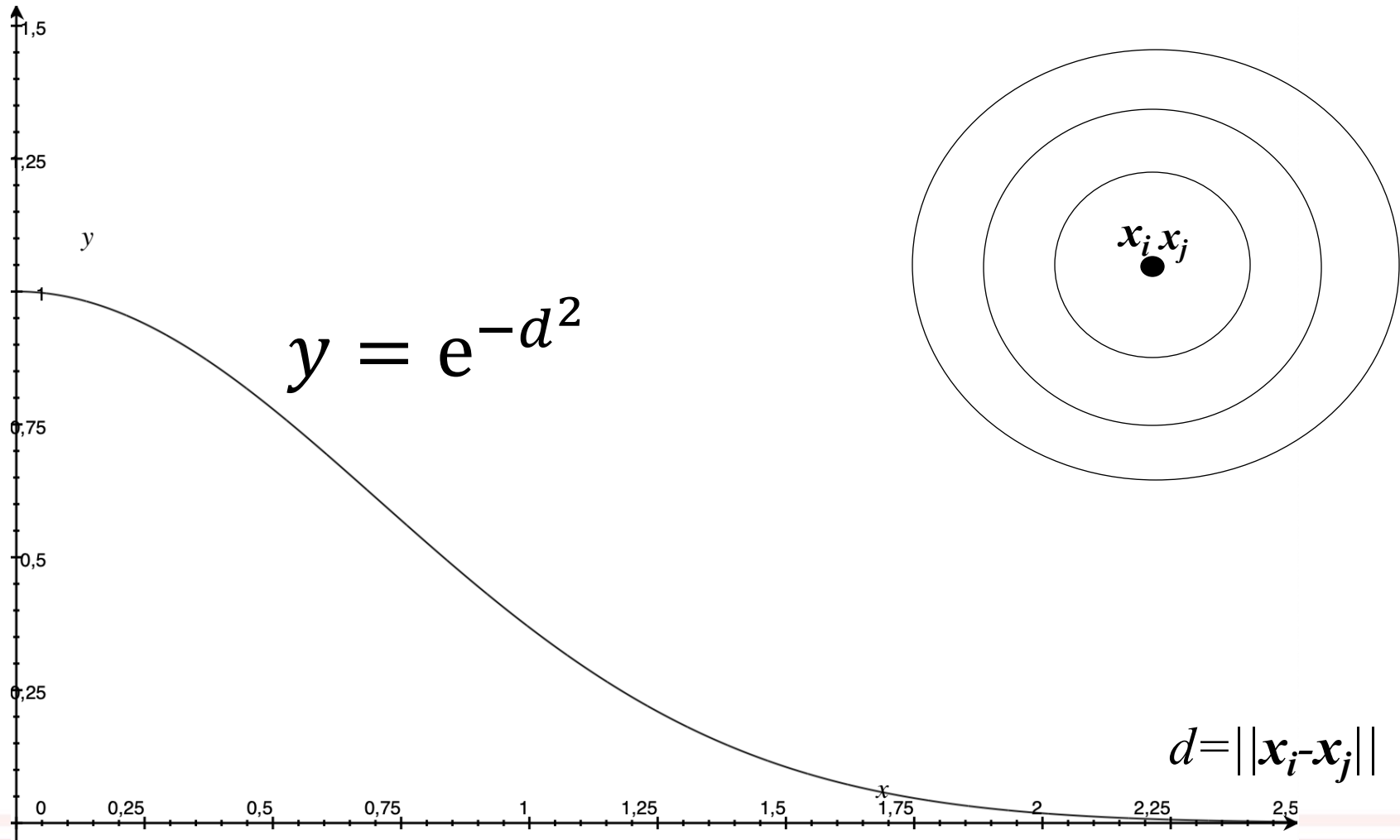
If the objective is to maximize the LOO accuracy, then the accuracy of $\mathbf{x}_i$ is the probability that other samples can correctly classify it:

$$p_i = \sum_{j \in \Omega_i} p_{ij},$$

Ω_i is the index set of samples with the same class label as $\mathbf{x}_i$.

Plot of the radial basis function

Radial basis function



Metric Learning

Neighbourhood Component Analysis (NCA)

□ Then, the LOO accuracy for the entire data set is:

$$\sum_{i=1}^m p_i = \sum_{i=1}^m \sum_{j \in \Omega_i} p_{ij}.$$

□ With $p_{ij} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|_{\mathbf{M}}^2)}{\sum_l \exp(-\|\mathbf{x}_i - \mathbf{x}_l\|_{\mathbf{M}}^2)}$ and $\mathbf{M} = \mathbf{P}\mathbf{P}^T$, the target of NCA is:

error $\nearrow$ $\min_{\mathbf{P}} 1 - \sum_{i=1}^m \sum_{j \in \Omega_i} \frac{\exp(-\|\mathbf{P}^T \mathbf{x}_i - \mathbf{P}^T \mathbf{x}_j\|_2^2)}{\sum_l \exp(-\|\mathbf{P}^T \mathbf{x}_i - \mathbf{P}^T \mathbf{x}_l\|_2^2)}.$

Solving this gives the distance metric matrix $\mathbf{M}$ that maximizes the LOO accuracy of the nearest neighbor classifier.

Metric Learning

- In addition to the supervised learning objectives, such as minimizing the error rate, we can also introduce **domain knowledge** into the optimization objective of metric learning.
- If we already know that **some samples are similar or dissimilar**, we can define a *must-link* constraints set $\mathcal{M}$ and a *cannot-link* constraints set $\mathcal{C}$, where $(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{M}$ indicates $\mathbf{x}_i$ and $\mathbf{x}_j$ are similar, and $(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{C}$ indicates $\mathbf{x}_i$ and $\mathbf{x}_j$ are dissimilar:

$$\begin{aligned} \min_{\mathbf{M}} \quad & \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{M}} \|\mathbf{x}_i - \mathbf{x}_j\|_{\mathbf{M}}^2 \\ \text{s.t.} \quad & \sum_{(\mathbf{x}_i, \mathbf{x}_k) \in \mathcal{C}} \|\mathbf{x}_i - \mathbf{x}_k\|_{\mathbf{M}} \geq 1 \\ & \mathbf{M} \succeq 0 \end{aligned}$$

- The constraint $\mathbf{M} \succeq 0$ ensures that $\mathbf{M}$ is positive semidefinite. The above problem minimizes the total distance between similar samples while keeping the individual distances between dissimilar samples no less than 1.

Metric Learning

- Different metric learning methods obtain good positive semidefinite symmetric distance metric matrices $\mathbf{M}$ based on different goals.
- When $\mathbf{M}$ is a low-rank matrix, by eigenvalue decomposition of $\mathbf{M}$, we can always obtain a set of orthogonal basis vectors, where $\text{rank}(\mathbf{M})$ is less than the original number of features d .
- Hence, the learning outcome of metric learning can derive a dimension-reduced matrix $\mathbf{P} \in \mathbb{R}^{d \times \text{rank}(\mathbf{M})}$ that can be used for dimensionality reduction.

Conclusion

- ❑ **k-Nearest Neighbor Learning**
- ❑ **Low-dimensional Embedding**
- ❑ **Principal Component Analysis**
- ❑ **Manifold Learning**
- ❑ **Metric Learning**

Thanks!
